Progressive Web Apps: The Web's Answer to Native Applications

Alberto Boffi
Connected Machinery & Digital Twin Platforms, Connect Reply, Milan, Italy
a.boffi@reply.it

Abstract

Progressive Web Applications (PWAs) represent a future direction in the field of software engineering, combining the widespread reach typical of web-based technologies with the enriched user experience more commonly linked with native apps. By means of a holistic evaluation of PWAs, based on a wide range of critical performance indicators, this study examines several dimensions such as technical effectiveness, hardware supportability, development process efficiency, and ecosystem maturity. As a result, this research clarifies the distinctive benefits of PWAs through rigorous direct comparisons with native and hybrid apps. The results based on this evaluation lead to an evidence-based framework capable of supporting developers, policymakers, and researchers in evaluating the strategic value of PWAs in various contexts.

Contents

1 Introduction **4**

2 Platforms **4**

3 Customization **6**

4 Performance **8**

4.1 Analysis 8

4.2 Memory Usage 10

5 Hardware and OS Access Capabilities **11**

5.1 Browser Support 11

5.2 Web APIs 12

6 Community Support **13**

7 Streamlined and Rapid Development Process **14**

7.1 Overview 14

7.2 Freedom from Backward Compatibility Constraints 16

8 Modularity and Integration **17**

9 Maturity and Future of the Technology **18**

10 Continuous Integration Support **20**

10.1 Automatic Testing 20

10.2 Static code analysis 21

11 SDLC Costs **21**

11.1 Development Costs 21

11.2 Maintenance costs 22

12 Integration with AI Tools **23**

12.1 Server-side integration 23

12.2 Client-Side Integration (In-Browser AI) 23

13 Accessibility **24**
13.1 Distribution and User Access 24
13.2 Web Content Accessibility Guidelines 25
13.3 Semantic HTML 26
13.4 Tools 28
13.5 Challenges 29
13.6 Search Engine Discoverability 29

14 Conclusion **29**

1. Introduction

Progressive Web Applications (**PWAs**) are types of web applications that utilize modern web technologies to provide an experience similar to that of native apps. These are built using universal web technologies, namely HTML, CSS, and JavaScript, but with additional functionalities typically associated with native apps, like offline capability, support for push notification, and ability to install from a device's home screen directly.

The core idea behind PWAs is to seamlessly combine the broad reach and accessibility of the web with the performance, engagement features, and app-like feel of native applications, without requiring distribution through traditional app stores.
The main advantages are the single codebase, instant dissemination and updates, no limitations placed by platform owners, no app store fees, better search engine optimization, and lower storage needs.

Despite growing excitement around PWAs, decision-makers often find themselves confused when evaluating them relative to native or hybrid offerings. While technical specs and successful pilot implementations do abound, few thorough reviews based on plainly stated Key Performance Indicators (KPIs) are available. This paper systematically examines PWAs against such KPIs, providing an analytical basis for understanding their suitability in various development scenarios. The analysis aims to provide practitioners, researchers, and stakeholders with a structured and evidence-based framework for evaluating PWAs in real-world decision-making.

2. Platforms

Platform and device reach capabilities are perhaps the largest and most intriguing benefits of PWAs.

Unlike native or compiled cross-platform solutions, PWAs do not, by their very nature, require platform-specific distribution pathways or app store approval for their basic deployment. Updates are frictionless and instant

for all, delivered directly from the web.

The benefits of this approach are the following:

- Single codebase: A single codebase to cover all of the environments. In other words, a single project and a single development team, fewer development time and cost.
- Instant global updates: Every update is out on every platform right away, with no user intervention required. This also reduces the issues of backwards compatibility, both across versions of the front-end application, and between front-end and back-end.
- Cheating app store restrictions: Ensures that the app is available everywhere and that there is no platform owner able to prohibit publication. No approval trail, no gatekeepers, no random blocking.
- No app store fees: No app store sign-up fee or commission charges.

There are, however, some disadvantages to it:

- A PWA relies on a web browser, so they can't be utilized in non-browser devices like smart TV, and their support is sparsely available on unsupported Chromium browsers.
- The existence and behavior of advanced PWA features may vary across browsers and operating systems.

The following is the list of supported platforms:

- Mobile: Android (excellent support), iOS (good and still enhancing support).
- Desktop: Windows, macOS, Linux, ChromeOS (where they support a near-native experience).
- Other environments: Minor Smart TVs, wearables, and embedded systems support.

Level of support and integration:

Android
Offers the most feature-rich PWA support. PWAs can be prompted to "Add to Home Screen," be accessed via the app drawer, be displayed full-screen, manage notifications, and integrate deeply with the OS.

iOS
Apple support within Safari has progressed a long way. PWAs can be installed to the home screen for a standalone, app-like experience. Caching and offline service workers are supported well. Push notifications in PWAs on iOS have existed since iOS 16.4, though feature parity with native app notifications or Android PWA notifications may still be in development.

Desktop
Large desktop browsers (most notably Chromium-based ones like Chrome and Edge, though also Safari on macOS and Firefox) support PWAs. This allows PWAs to run in their own sandboxed windows, to show taskbar/-dock icons, and to behave very much like traditional desktop applications.

The Apple limitation comes from the reality that Chrome on iOS relies on the WebKit engine. Apple is affected in the European Union by the **Digital Markets Act**, which prohibits it from excluding competing browsers. This would allow Google to implement the Chrome browser on the Chromium engine even on Apple gadgets, with PWAs maintaining the same features as they do on Android.

3. Customization

A Progressive Web App's potential for customization is directly related to the power and flexibility of the open web platform. Built with core web technologies (HTML, CSS, and JavaScript) PWAs give developers almost total control over the look, feel, and behavior of the app on all devices.

As opposed to platforms that may prescribe their own UI toolkit or abstract

the render layer away, a PWA renders directly into the browser engine. This allows developers direct access to the Document Object Model (DOM) and pixel-perfect, fine-grained design control. This open, standards-based model allows:

- Total Design Freedom: Developers aren't limited to a set of predefined components. They can create fully custom user interfaces that perfectly reflect a brand's identity, include any third-party UI library, or develop their own component ecosystems unhindered by platform constraints. Or they can take advantage of popular and highly extensible CSS frameworks like Tailwind CSS, Bootstrap, or custom SCSS to accelerate development while still maintaining the ability to override any style.

- Advanced Responsive Design: PWAs excel at adapting to different form factors due to new CSS techniques like Media Queries, Flexbox, and Grid. Interfaces can be resized dynamically and automatically for any screen size, from mobile to tablet to desktop, so the experience is always ideal everywhere.

- Rich Animations and Transitions: Using CSS transitions, keyframe animations, and powerful JavaScript animation libraries (e.g., GSAP, Framer Motion), it is possible to create smooth, interactive, and advanced animations that enhance the user experience and give the application a professional, polished look.

- Theming and Dynamic Styles: New CSS capabilities, in particular Custom Properties (variables), have made it simple to establish robust theming systems. A developer can define a global design palette (colors, fonts, spacing) that can be applied consistently and changed dynamically, which makes "dark mode" accessibility adjustments, or user-selectable themes trivial to implement.

- Deep Experience Control: Customization extends beyond the UI. The service worker, a core PWA building block, enables developers to exactly control caching mechanisms, define sophisticated offline behavior, and manage push notifications, all resulting in an extremely

tailored and performant user experience that feels integral to the device.

4. Performance

4.1 Analysis

Unlike compiled frameworks, PWAs execute within the browser environment, and therefore their performance will be as good as the browser engine (e.g., Chromium or WebKit) and hardware available on the device.

Existing browser engines offer high-performance JavaScript execution along with hardware-accelerated rendering that enable responsive and smooth user interfaces, especially for contemporary devices.

Key performance factors influencing PWAs are:

- Loading Performance & Service Workers: One of their strong points is loading performance, particularly on return visits. Service workers are JavaScript scripts that run in the background, without reference to the web page. They allow for advanced caching mechanisms like pre-caching of app assets (HTML, CSS, JavaScript, images, API responses). This allows resources to be served directly from the cache, providing lightning-fast loading times and reliable offline access.
- Installation Performance: PWAs install instantly over HTTPS directly in the browser, zero app store latency, and substantial first time downloads. The storage footprints is also tremendously low (download on-demand), hence also the download time is negligible.
- Runtime Performance: Depending on good JavaScript, UI complexity, and browser rendering. Libraries like React, Angular, and Vue.js provide performance utilities for UIs (e.g., virtual DOM or incremental DOM). Browsers like Google Chrome provide strong mechanisms for sustaining smooth runtime performances.

- Responsiveness and Smoothness: Achieved through CSS animations/transitions, which are typically hardware-accelerated, providing best performances.

Some drawbacks:

- Although, thanks to modern browser technology, the performance of a PWA is practically indistinguishably similar to that of a native app for most apps available for purchase, they won't necessarily match raw performance of completely native or compiled applications for graphically intensive applications like real-time 3D gaming or where tight, low-level hardware integration or deep background processing is required.
- The UI smoothness is ensured through hardware-accelerated CSS transitions and rules. Native applications use the operating system's API and OS directly for gestures and animations. In PWAs, UI changes are routed through the browser's rendering engine to be forwarded to the operating system, resulting in overhead. This results in very little touch and animation latency and less control over performance, though slight, which under certain conditions becomes noticeable by the user.

To monitor and optimize performances, Google Chrome offers **Google's Lighthouse** tool (Figure 5), in which developers can audit PWAs, identify performance bottlenecks, and check compliance with performance best practices, accessibility guidelines, and PWA checklist requirements.

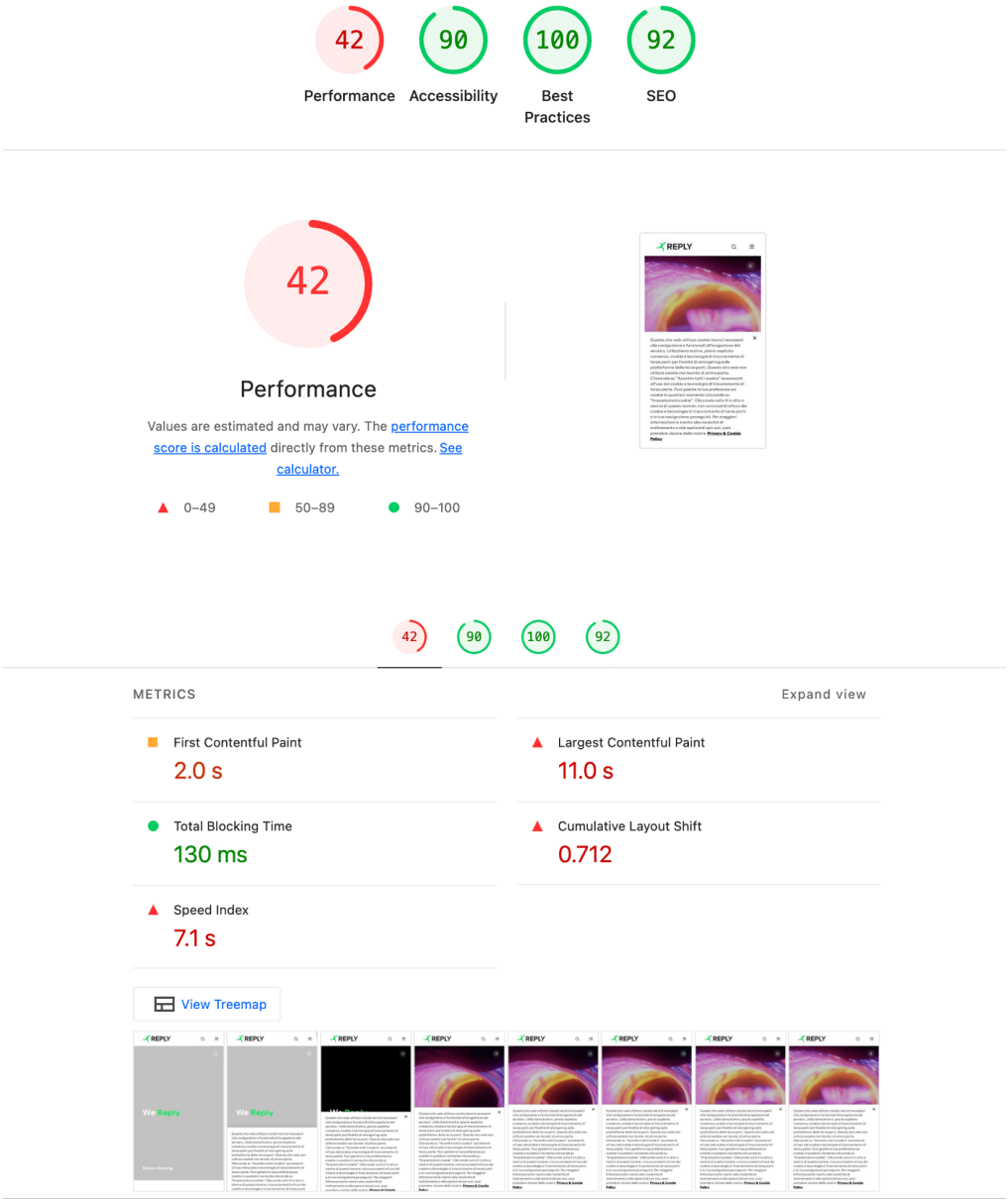


Figure 1: Performance highlighting from Google Lighthouse tool.

4.2 Memory Usage

Apart from time, the most valuable asset of PWAs is space used. Because of the "download on demand" capability, PWAs are much smaller than native applications. This saves valuable device storage space, leading to faster loading times, improved user experience, reduced burden on user data and device resources.

5. Hardware and OS Access Capabilities

When it comes to Progressive Web Apps, the ability to access the operating system is one of the most frequently asked questions. Unlike native apps, where the app directly interfaces with the operating system, which in turn interfaces with the hardware (Figure 2), here we have an additional layer represented by the browser (Figure 3).

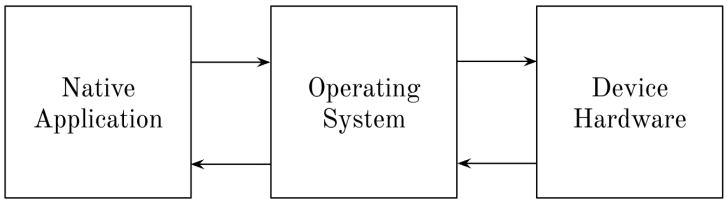


Figure 2: Native app access to device hardware.

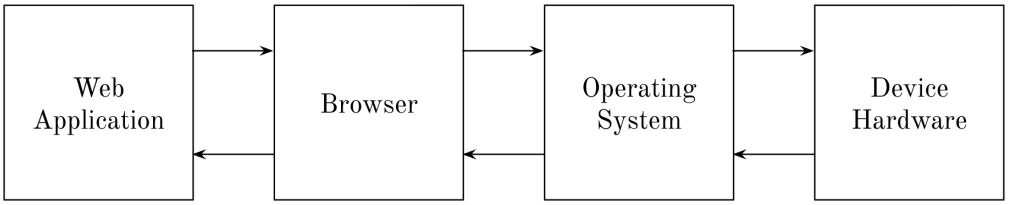


Figure 3: Web app access to device hardware.

5.1 Browser Support

A necessary premise is the distinction between browsers. Actually, in this sense, it makes less sense to speak of "browser" rather than rendering engine, and, therefore, operating system.

On Android, browsers use **Chromium** (developed by Google) as their rendering engine, while on iOS they use **WebKit** (created by Apple). Chromium is the rendering engine that offers greater support for PWAs, which makes Android the perfect operating system for PWAs. Conversely, WebKit is more limiting, so much so that it restricts even Google Chrome on the iOS operating system.

This situation seems to be partially resolvable in the European Union

thanks to the **EU Data Act**, a regulation designed to foster a more competitive data market in Europe. This regulation could impact Apple by preventing it from limiting the capabilities of external browsers, effectively allowing Google to build its browser on the Chromium engine even on Apple devices.

5.2 Web APIs

Since web apps have an additional step before accessing the operating system, a method is needed for the app to communicate with the browser to indicate which OS resources, and therefore hardware, it requires. This method is provided by Web APIs (Figure 4). Web APIs can extend browser functionalities in many ways, accessing Bluetooth, the filesystem, Battery Status, and much more. Some are now supported by all browsers, such as those for camera access (Media Capture and Streams API) or those for file system access (File System API), while others are newly developed and in the early stages, such as those for NFC usage (NFC API).

What we cannot do is list all existing Web APIs, nor how much they are supported by browsers on various operating systems. This is because they are rapidly expanding and are ensuring a prosperous future for PWAs. For a reference, the best documentation to consult is Mozilla's: https://developer.mozilla.org/en-US/docs/Web/API.

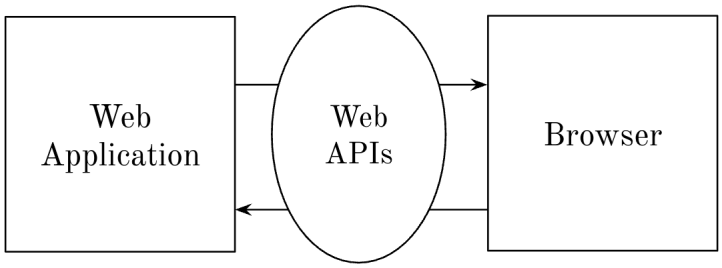


Figure 4: The role of Web APIs in app-browser communication.

6. Community Support

One of the biggest benefits of PWAs is that they enjoy community support. PWAs are part of the larger web development community, which is the largest, most mature, and most active community in the software world.

The **World Wide Web Consortium** (**W3C**) is the principal international organization that deals with *Web standard* setting. It was established by Tim Berners-Lee in 1994, and its mission is to ensure the long-term growth and interoperability of the Web.

These standards are the foundations for technologies like HTML, CSS, XML, and accessibility guidelines (WCAG). The standards are crucial to:

- Interoperability: Enabling web content to work on different browsers and devices.
- Accessibility: Making the Web accessible to all, including people with disabilities.
- Evolvability: Allowing the Web to adapt without sacrificing existing content.
- Openness: Preventing single-company control of foundation web technologies.

Standards are authored by a consensus-oriented, collaborative process involving its global member organizations, staff, and the public.

Further, the support landscape is wide-ranging and encompasses numerous channels:

- Authoritative Documentation: MDN Web Docs (Mozilla) and web.dev (Google) provide comprehensive, in-depth, and current guidance, tutorials, and best practices.
- GitHub: Endless library of open-source tools, libraries, and frameworks (e.g., Workbox for service workers, Vite, Next.js), and end-to-end PWA examples.

- Package Registries: The npm registry is the largest software registry globally and provides immediate access to millions of reusable code packages for any conceivable functionality.
- Developer Forums: Stack Overflow has millions of questions and answers in web-related tags (e.g., progressive web apps, service-worker, javascript). Millions of other forums, newsletters, and subreddits (e.g., r/webdev) provide daily activity.
- Conferences and Events: Major events like Google I/O and Chrome Dev Summit tend to have PWA-focused content and announcements.

Since PWAs are based on open standards, they are supported by major browser vendors (Google, Mozilla, Microsoft, and Apple) and stay up to date with new specs. The community also engages regularly in browser APIs, tooling, and best practices.

The main cons, though, are:

- The sheer number of choices (frameworks, tooling, libraries) itself can lead to decision paralysis and fragmentation of toolchains.
- Even with browser compatibility being so much improved nowadays, occasionally developers must still account for tiny discrepancies or back-end feature support, particularly between leading browser engines (e.g., WebKit on iOS vs. Chromium).
- The JavaScript ecosystem's high pace can lead to orphaned packages or the need to handle compatibility carefully between dependencies.

7. Streamlined and Rapid Development Process

7.1 Overview

Progressive Web App evolution draws its foundations from simplicity and speed, mainly because it also draws upon the complete set of skills surrounding web development.

The lifecycle of development follows a much faster pace compared to native apps, due to several factors:

- Benefitting from Proven Frameworks and Instruments: Public frameworks like Angular or React have official and community-based templates available, which can help reduce the setup process by including essential components of a PWA by default.

- Rapid Iteration and Troubleshooting: Modern build tools and libraries like Vite, Webpack, and Angular support fast build times and provide features like Hot Module Replacement (HMR), thus providing instant feedback in the browser environment. Browser development tools also provide extensive inspection and debugging capabilities specially designed for Progressive Web Applications, featuring functionalities for inspecting the manifest, tracking the lifecycle of the service worker, and allowing offline emulation.

- Low Technical Entry Barrier: The first core technology barrier to market entry is that teams with HTML, CSS, and JavaScript skills do not need to be familiar with other programming languages or frameworks like Kotlin or Swift. This makes it easier to enter the market, increases the pool of talent available, and encourages the accelerated onboarding of developers.

- Instant Deployment: PWAs are distributed via the internet, thus eliminating the long procedures related to compilation, packaging, and submission to app stores. Updates are deployed by simply uploading new files into a server, which makes them available instantly to users and allows for much faster iteration cycles.

- Unified Codebase: One codebase can be used to support all devices running modern browsers, including Android and iOS smartphones, and desktop systems such as Windows, Mac, and Linux, thereby making it easier to develop quickly and reduce upkeep costs.

This becomes a major limitation when high-level integrated functionality, like direct Bluetooth connection and sophisticated background processing,

becomes a requirement, as dependency upon browsers can add constraints or unpredictability, thereby hindering the release schedule.

7.2 Freedom from Backward Compatibility Constraints

A very valued advantage of Progressive Web Applications to developers, in particular, is that there will be no back-compatibility issues. That means everything from new releases will be immediately available to the final client.

The main strength of this approach lies in its guarantee that any security patches or necessary functionality will be immediately applied by the receiving end user.

The second major advantage relates to synchronization across the front-end and back-end systems. The addition of virtually any newly created feature requires changes to be implemented in both the front-end and back-end systems as well. The lack of necessary adjustments in either end can lead to faulty operation of the application. For native applications, this problem can be overcome by implementing several mechanisms:

- The necessity to update the application to allow its continued use represents a significant barrier to assessing the overall experience of the final user.
- The extremely efficient nature of the back-end lends itself to situations where changes in the front end go unimplemented, allowing the program to run smoothly and then requiring additional input from the back end. In addition, this process involves having additional logic and code that might not be considered necessary.
- The approach requires preconfiguration of the front end to be compatible with previous iterations of the back end. This requires the first release to be only of the front end, thus giving users ample time to update the application prior to release of the following back end version. This strategy, however, which embeds further features and intricacies in the front end, does not guarantee all users have made

requisite application upgrades, thereby injecting a probable risk of issues after release of the back end.

Clearly, no simple and straightforward solution has been created yet. Progressive Web Apps tackle this challenge at its root level. This feature represents one of the most critical, yet often overlooked, values of Progressive Web Apps and web applications in general.

8. Modularity and Integration

PWAs excel in modularity and integration, thanks to the open, component-driven nature of the modern JavaScript ecosystem. This allows for scalable architectures and rapid experimentation for projects of any size.

The possibilities of integration are vast:

- *npm* Ecosystem: One main feature of modularity is embodied by npm (Node Package Manager), which happens to be the world's largest software registry. Developers can build applications by combining modular packages, from simple UI component libraries (like Material UI) to utilities meant to help manage state (e.g., Redux) to analytics, form validating, and internationalizing.

- Framework-Intrinsic Modularity: User interface frameworks like Angular, React, or Vue necessarily organize applications in terms of independent and reusable components. This property fits nicely within a modular paradigm, ensuring a clear split of responsibilities.

- Fluent API and Service Integration: Progressive Web Apps can integrate easily with different third-party services using standard protocols like REST, GraphQL, or WebSockets. They can further integrate themselves into major platforms like Firebase, Stripe, or Auth0 using their respective JavaScript Software Development Kits (SDKs). Developers can now enjoy near-native functionality by leveraging strong JavaScript APIs like WebAssembly (for performance-critical code), WebRTC (for real-time communication), or IndexedDB (for complex client-side data storage).

- Service Worker's Modularity: Service workers are defined by their modular, extensible architecture. Google suggests libraries like Workbox, which consists of a set of reusable modules aiming to abstract and simplify complex functionalities dealing with caching, routing, and background synchronization.

However, a number of issues arise:

- The requirement for careful dependency management to keep packages up to date.
- Ensuring libraries and APIs' interoperability within multiple web browsers and execution environments. The wide range of tools available can periodically lead to fragmentation or so-called "decision fatigue".

9. Maturity and Future of the Technology

Progressive Web Applications are not limited to a particular framework; instead, they are built on open web standards set by entities like W3C and supported by browser vendors like Google, Mozilla, Apple, and Microsoft. This feature makes PWAs one of the most cutting-edge and secure technologies within application development.

The underlying technologies of HTML, CSS, and JavaScript have been used for a long time. Progressive Web Applications were officially sanctioned in the mid-2010s and subsequently adopted by leading brands like Twitter, Starbucks, Pinterest, and Uber for both desktop and mobile usage. This ecosystem has proven stable and has been thoroughly tested, especially with regards to key pieces like service workers and web app manifests, which have received widespread backing on multiple web browsers. Moreover, libraries and tools like Workbox, Lighthouse, and PWA Builder remain actively supported and developed by major stakeholders in the industry.

Pros:

- Backed by open standards and major industry players, ensuring longevity.
- Based on strong and cutting-edge underlying technologies.
- A clear roadmap for expanded capabilities via new browser APIs.

Challenges

- Variability in feature support across browsers, especially on iOS where some features have historically lagged.
- A fragmented tooling ecosystem may affect consistency compared to a single-framework approach.

The future of PWAs is intrinsically tied to the evolution of the web platform itself, which is advancing rapidly. Here are some examples of innovation drivers:

Project Fugu
A key driver is the Web Capabilities initiative (codenamed Project Fugu), a Google-led effort to close the capability gap between native and web by providing secure access to features like the File System Access API, Web Bluetooth, and advanced hardware integration.

Google's underlying principle is to extend the ability of the web by utilizing innovative web APIs, which allow web programs to access hardware and functionality heretofore reserved to native programs.

WebAssembly
WebAssembly, or Wasm, is an open-standard framework of web technologies to run programs written in a variety of programming languages other than JavaScript, like C, C++, and Rust, in web environments. It defines an optimized binary format and its respective text format, aiming to provide performance similar to native execution of machine code. Initially proposed to improve elemental parts of web application components, WebAssembly has transformed to allow developing and running web applications based

on a variety of programming languages, thereby extending its purpose beyond just supporting JavaScript. Development of this technology happens within the auspices of World Wide Web Consortium (W3C) and opens itself to contributions from many organizations, including Mozilla, Microsoft, Google, and Apple.
The increasing maturity of WebAssembly allows more performance-intensive applications to run efficiently in the browser, significantly expanding the scope of what PWAs can achieve.

App Store Integration Tools
Trusted Web Activities, created by Google, and Capacitor, built by Ionic, allow PWAs to be packaged to different application marketplaces, such as the Google Play Store, Apple Store, and Microsoft Store, thereby offering hybrid distribution approaches.

10. Continuous Integration Support

Progressive Web App development environment is furthered by a thorough set of modern web codes analysis and testing tools, a significant portion of which have a deep heritage, solid usage, and significant flexibility.

10.1 Automatic Testing

The comprehensive methodology of assessment can be attained by implementing accepted tools:

- Unit Testing: Libraries like Jest, Mocha, and Vitest are utilized for testing individual JavaScript functions.
- Component and Integration Testing: Tools like Testing Library (for React, Vue, etc.) and Cypress are ideal for testing UI components in isolation and their interactions.
- End-to-End (E2E) Testing: Playwright, Selenium WebDriver, and TestCafe all make it possible to automate actual web browsers to mimic extensive user interactions.

- Performance and PWA Audits: Google Lighthouse is the standard for auditing performance, accessibility, and PWA-specific criteria like installability and offline support.

These tools can simulate offline conditions, verify service worker behavior, test installation flows, and measure responsiveness, crucial for ensuring a PWA behaves reliably.

10.2 Static code analysis

PWAs benefit from a robust JavaScript and TypeScript ecosystem for static analysis:

- Linters and Formatters: ESLint for code linting and rule enforcement, typically paired with Prettier for consistent formatting.
- Type Verification: TypeScript provides static type safety, which detects a broad class of errors before run time.
- Security and maintainability analysis: A thorough analysis can be performed using tools like SonarQube or CodeQL created by GitHub to flag potential security vulnerabilities and issues regarding maintainability.

These tools are easily integrated into CI/CD pipelines (e.g., GitHub Actions, GitLab CI), ensuring continuous quality assurance.

11. SDLC Costs

The financial framework which facilitates the upkeep and development of a PWA gives a significant strategic advantage, especially because of its basis in established web development practices. This approach typically yields enormous savings in terms of cost as well as time compared to native application development.

11.1 Development Costs

The primary driver of cost savings is the adoption of a shared codebase. Instead of spending resources on separate development teams for iOS and

Android, a single team of web developers can produce one application that runs flawlessly across all platforms. This consolidation dramatically reduces both development time and the complexity inherent in project management.

Furthermore, PWAs are built with ubiquitous technologies like HTML, CSS, and JavaScript, along with popular frameworks such as React, Angular, or Vue. This means there is a vast pool of developer talent available, making it easier and often more cost-effective to staff a project compared to finding specialized native mobile developers.

Another key aspect involves avoiding traditional application distribution stores. This strategy avoids the monetary costs involved with app store submission costs (e.g., the recurring cost of the Apple Developer Program) and, more importantly, avoids the typically long and, at times, uncertain review and approval processes. This reduces the time to market and frees up resources otherwise spent on coordinating submissions to such platforms.

11.2 Maintenance costs

The effectiveness of the PWA model becomes more pronounced in long-term support scenarios. Upgrade processes become streamlined and cost-effective by virtue of:

- No app store approvals are needed, enabling continuous deployment and rapid iteration cycles.
- There's an independent version of the program available constantly on-line, enabling instant execution of corrective patches or additional features to all users.
- There is no necessity to manually update users' software to gain access to the newest version.

In the long term, it creates less operational complexity, more efficiency in quality testing and assurance protocols, and fewer infrastructure costs.

12. Integration with AI Tools

PWAs are exceptionally well-suited to integrate with modern AI and machine learning tools, thanks to their foundation in the open and flexible web ecosystem. Whether using cloud-based AI APIs or running models directly in the browser, PWAs can leverage both server-side and client-side AI solutions without major architectural constraints. Developers can mix server-side and client-side approaches to best fit their needs.

12.1 Server-side integration

This is the most straightforward and powerful method for embedding advanced AI. PWAs can easily call external APIs from major cloud providers for a wide range of tasks. This includes:

- Generative AI and NLP: Calling services like OpenAI API, Google Cloud AI (Gemini, Vertex AI), Microsoft Azure AI, or AWS AI Services for text generation, translation, vision, or speech processing.
- Conversational User Interfaces: Including chatbot solutions such as Google Dialogflow or Rasa.
- Personalization: Enabling recommendation systems and personalized search by utilizing REST or GraphQL interfaces.

12.2 Client-Side Integration (In-Browser AI)

For real-time, low-latency, and privacy-focused applications, PWAs can run AI models directly in the browser using TensorFlow.js, ONNX.js, or WebAssembly-based runtimes. Because a PWA's service worker can cache the AI model, these features can even work offline. Furthermore, PWAs can seamlessly combine client-side AI with other modern browser APIs for powerful interactive experiences. Two of the most common examples:

- WebRTC and camera/microphone functionality: Used for real-time audio and video data processing.
- Web Speech API: Used to enable voice commands and text-to-speech (TTS) capabilities.

Standards like the **Web Neural Network API** (**WebNN**) are expected to greatly enhance these features through the delivery of hardware-acceleration for devices' integrated chips for artificial intelligence to web applications.

The limitations of both approaches are the following:

- Performance Constraints of Browsers: Intricate or large-scale AI models can run slowly on the client-side, and even the GPU-based acceleration is quite limited compared with native applications.
- Offline Use Requires Setup: Caching models for offline AI functionality requires careful implementation of service workers and storage strategies. The dependency of high-end artificial intelligence on a server requires an internet connection, which can lead to related server costs.

13. Accessibility

Inherent accessibility of PWAs represents a major strength, deriving from their base in open web standards and the large body of knowledge about web accessibility developed over the years. Native application combinations generally require separate development and accessibility testing for each separate platform (e.g., Android, iOS, desktop), whereas PWAs apply tried-and-true technology that is more accessible per se and platform-agnostic.

13.1 Distribution and User Access

With respect to distribution and user accessibility, PWAs provide significant advantages over native applications:

- No installation required: Users can access a PWA directly from any browser, on any device, without downloading or installing anything.
- Neutralization of app store dependence: Lack of app store gatekeeping neutralizes any need for approval procedures and eliminates restrictions set through marketplace guidelines.

- Immediate accessibility and revisions: Changes take effect immediately, and changes are updated at once across all user groups, ensuring improvements or changes are easily accessible.
- Cross-device availability: Whether on desktop, tablet, or mobile, users can access the same application with consistent accessibility support.

13.2 Web Content Accessibility Guidelines

PWAs, in their very nature as web applications, follow the basic principles that ensure accessibility for people with disabilities, such as visual impairment, hearing disability, mobility impairment, and cognitive impairment. The major goal is to make the application perceivable, operable, understandable, and robust, in line with set standards like the **Web Content Accessibility Guidelines** (**WCAG**).

The WCAG are a widely supported standard set by the World Wide Web Consortium (W3C). The main goal is enabling websites' accessibility for disabled people with various impairment types like visual impairment, auditory impairment, physical impairment, speech impairment, or cognitive impairment without compromising the needs of other user groups like older adults, individuals with temporary restrictions, and device-diverse-interfacing users.

WCAG is built on four core principles (POUR):

- Observable: Students can see the data being displayed.
- Functional: Users can interact with the interface and move within it.
- Understandable: Information and operation are clear and predictable.
- Robust: Content works reliably across different technologies.

The guidelines below are organized along Measurable Success Criteria defined across three levels of adherence:

- Level A (Minimum)

- Level AA (Target and most common legal requirement)
- Level AAA (Highest)

Key versions include WCAG 2.0, 2.1, and the current 2.2, with 3.0 in development. Adhering to WCAG is crucial for inclusivity, legal compliance (e.g., ADA, European Accessibility Act), improved user experience, SEO benefits, and reputation.

Google Lighthouse can help also to evaluate the accessibility of the PWA, giving advices on how to correct the problems that have been encountered. Not following the Web Content Accessibility Guidelines (WCAG) can lead to legal consequences, as well as possible financial penalties (Figure 5).

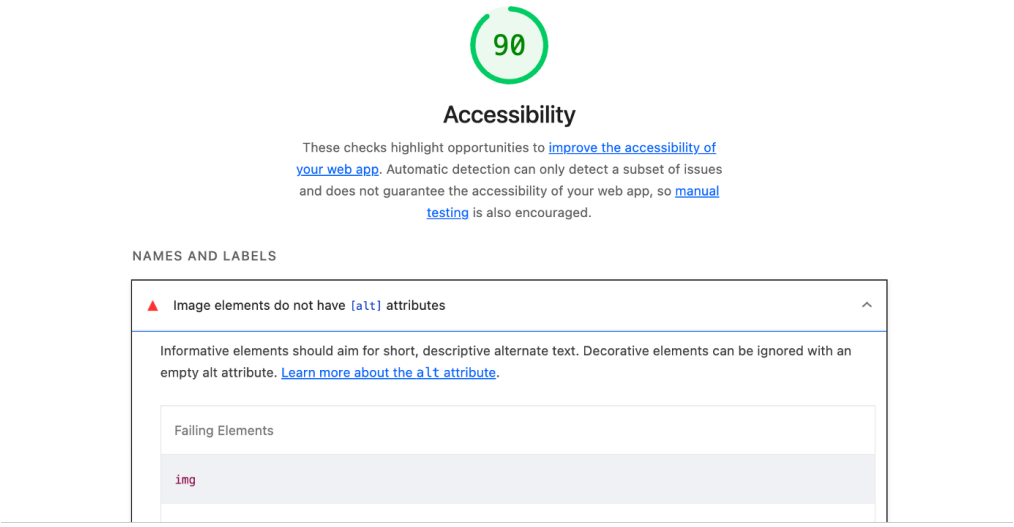


Figure 5: Accessibility overview from Google Lighthouse tool.

13.3 Semantic HTML

Semantic HTML is the bedrock of an accessible PWA. Using correct, semantic HTML tags provides an inherent structure and meaning. Assistive technologies, such as screen readers, rely on this semantic structure to interpret the page and provide a coherent experience for the user.

ARIA (**Accessible Rich Internet Applications**) is a set of HTML

attributes aimed at promoting the accessibility of complex internet applications and dynamic web pages for disabled individuals relying on assistive technologies like screen reader software. It overcomes the shortcomings of traditional HTML in the communication of semantic information needed for the efficient communication of complex interactive elements like tailored tabs and carousels. It is made possible with the use of:

- Roles: Defining what an element is (e.g., `role="button"`).
- States: Describing an element's current condition (e.g., `aria-expanded="true"`).
- Characteristics: Providing additional details about a specific element (e.g. `aria-label="Search"`).

Crucially, ARIA should only be used when native HTML elements cannot provide the necessary semantics. It ensures assistive technologies can correctly interpret and convey interactive elements, improving the user experience for everyone, especially those relying on keyboard navigation.

Other Factors to Consider:

- Keyboard Focus Management: Interactive elements within a PWA should be accessible and usable only through keyboard input alone. In the Single-Page Application (SPA) design commonly used by many PWAs, developers should carefully control the user's "focus" whenever a new view is updated or a modal dialog is displayed. Focus should be programmatically changed to the newly exposed content. Otherwise, keyboard or screen reader users may be left behind on the previous screen with no awareness of any changes in the content.
- Web Best Practices: Progressive Web Applications PWAs must adhere to basic accessibility standards, which involve providing proper color contrast between text and background, providing textual descriptions (alt attributes) for any important images, and using a responsive design that supports scaling of content while preserving the integrity of the important layout and key details.

- OS-Level Integration: Even the PWA's manifest.json file plays a role. The `name` and `short_name` properties are what the operating system's screen reader (e.g., VoiceOver on iOS, TalkBack on Android) will announce when a user navigates to the PWA's icon on their home screen.

13.4 Tools

Most modern front-end frameworks used for PWA development (e.g., React, Vue, Angular) include built-in support or best-practice patterns for accessible components. In addition, many open-source UI libraries (e.g., Material UI, Radix UI) provide prebuilt, accessibility-compliant components out of the box.

PWAs also benefit from advanced accessibility features provided by modern browsers, such as:

- Extensive compatibility with screen reader technologies (i.e., NVDA, JAWS, VoiceOver)
- Settings with high contrast and little motion.
- Zoom and font scaling
- Keyboard and focus management

Testing accessibility in PWAs is made easier through powerful and widely used tools such as:

- Accessibility evaluation using Lighthouse (bundled with Chrome DevTools).
- axe-core (for automated testing).
- VoiceOver, NVDA, and TalkBack (for manual testing).
- ChromeLens and Accessibility Insights extension developers.

13.5 Challenges

- Requires Developer Diligence: Accessibility is not automatic. It depends entirely on developers having the knowledge and taking the conscious effort to implement it correctly.
- SPA Complexity: In single-page PWAs, developers must manually manage focus and announce route changes to assistive technologies to prevent user disorientation.
- Risk with Custom Components: Over-reliance on non-semantic elements (like `<div>`s for buttons) can easily lead to an inaccessible experience if ARIA attributes are not implemented meticulously and correctly.

13.6 Search Engine Discoverability

A key distinction from native apps is that PWAs live on the web, meaning they are discoverable by search engines like Google and can be shared with a simple link. This allows for organic growth via **Search Engine Optimization** (**SEO**).

14. Conclusion

Progressive Web Applications are a viable option over native and hybrid development. They provide seamless access, reduced costs for development and maintenance, and faster deployment times. They support offline functionality, push notifications, and can be installed directly from a browser, making them akin to both web and native applications. While PWAs might not be able to offer the same level of performance or integration with hardware as native applications, these applications perform satisfactorily in scenarios where reach, flexibility, and cost matter most. As browser capabilities continue to advance and support for various platforms become more universal, PWAs are going to have a significant impact in app development in the future, especially for enterprises seeking solutions that are scalable, simple to maintain, and are not platform-specific.

References

Ater, T. (2017). *Building progressive web apps: Bringing the power of native to the browser*. " O'Reilly Media, Inc."

Azioni consentite | Capabilities | Chrome for Developers — developer.chrome.com [[Accessed 10-08-2025]]. (n.d.).

Biørn-Hansen, A., Majchrzak, T. A., & Grønli, T.-M. (2017). Progressive web apps: The possible web-native unifier for mobile development. *International Conference on Web Information Systems and Technologies*, 2, 344–351.

Caldwell, B., Cooper, M., Reid, L. G., Vanderheiden, G., Chisholm, W., Slatin, J., & White, J. (2008). Web content accessibility guidelines (wcag) 2.0. *WWW Consortium (W3C)*, 290(1-34), 5–12.

Fithriyaningrum, D., Kusumawardhani, S. S., Wibirama, S., et al. (2021). Analisis aksesibilitas website berdasarkan web content accessibility guidelines (wcag): Ulasan literatur sistematis (an analysis of website accessibility based on web content accessibility guidelines (wcag): A systematic literature review). *JURNAL IPTEKKOM (Jurnal Ilmu Pengetahuan & Teknologi Informasi)*, 23(1), 79–92.

Fortunato, D., & Bernardino, J. (2018). Progressive web apps: An alternative to the native mobile apps. *2018 13th Iberian Conference on Information Systems and Technologies (CISTI)*, 1–6.

Heričko, T., Šumak, B., & Brdnik, S. (2021). Towards representative web performance measurements with google lighthouse. *Proceedings of the 2021 7th student computer science research conference*, 39.

Hume, D. (2017). *Progressive web apps*. Simon; Schuster.

Isa, W. A. R. W. M., Suhaimi, A. I. H., Ariffrn, N., Ishak, N. F., & Ralim, N. M. (2016). Accessibility evaluation using web content accessibility guidelines (wcag) 2.0. *2016 4th international conference on User Science and Engineering (i-USEr)*, 1–4.

Majchrzak, T. A., Biørn-Hansen, A., & Grønli, T.-M. (2018). Progressive web apps: The definite approach to cross-platform development?

Malavolta, I., Procaccianti, G., Noorland, P., & Vukmirovic, P. (2017). Assessing the impact of service workers on the energy efficiency of progressive web apps. *2017 IEEE/ACM 4th International Confer-*

ence on Mobile Software Engineering and Systems (MOBILESoft), 35–45.

McGill, T., Bamgboye, O., Liu, X., & Kalutharage, C. S. (2023). Towards improving accessibility of web auditing with google lighthouse. *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, 1594–1599.

MDN Web Docs — developer.mozilla.org [[Accessed 10-08-2025]]. (n.d.).

Mishra, R. S. (2016). Progressive webapp. *International Research Journal Of Engineering and Technology (IRJET)*, 2395–0056.

Roumeliotis, K. I., & Tselikas, N. D. (2022). Evaluating progressive web app accessibility for people with disabilities. *Network*, 2(2), 350–369.

Steiner, T., LePage, P., Nattestad, T., McClelland, R., Russell, A., & Ng, D. (2020). From fugu with love: New capabilities for the web. *Companion Proceedings of the Web Conference 2020*, 135–138.

Tandel, S., & Jamadar, A. (2018). Impact of progressive web apps on web app development. *International Journal of Innovative Research in Science, Engineering and Technology*, 7(9), 9439–9444.

Thakur, P. (2018). Evaluation and implementation of progressive web application.

W3C — w3.org [[Accessed 10-08-2025]]. (n.d.).

Wang, W. (2021). Empowering web applications with webassembly: Are we there yet? *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 1301–1305.

Wargo, J. M. (2020). *Learning progressive web apps*. Addison-Wesley Professional.

Web Content Accessibility Guidelines (WCAG) 2.1 — w3.org [[Accessed 10-08-2025]]. (n.d.).

Web Neural Network API — w3.org [[Accessed 10-08-2025]]. (n.d.).

WebAssembly — webassembly.org [[Accessed 10-08-2025]]. (n.d.).